



UTM
UNIVERSITI TEKNOLOGI MALAYSIA

FACULTY OF COMPUTING

SEMESTER 2 2025/2026

SECP3133 – HIGH PERFORMANCE DATA PROCESSING

SECTION 2

PROJECT 2

REAL TIME SENTIMENT ANALYSIS USING APACHE SPARK AND KAFKA

LECTURER: DR. SEAH CHOON SEN

GROUP ABC

STUDENT NAME	MATRIC NO
JOANNE CHING YIN XUAN	A23CS0227
LIM YU HAN	A23CS0241
CHUA JIA LIN	A23CS0069
EVELYN GOH YUAN QI	A23CS0222

Presentation Video Link: <https://youtu.be/DQB8pdiOjMM>

TABLE OF CONTENTS

1.0 Introduction	2
1.1 Project Background	2
1.2 Problem Statement	3
1.3 Objectives	3
1.4 Project Scope	4
2.0 Data Acquisition and Preprocessing	4
2.1 Data Source Description	4
2.2 Data Collection Method	5
2.3 Data Cleaning Steps	6
2.4 Final Dataset Description	7
3.0 Sentiment Model Development	7
3.1 Feature Engineering	8
3.1.1 Text Preprocessing	8
3.1.2 TF-IDF Feature Extraction	8
3.2 Models Selection	9
3.2.1 Multinomial Naive Bayes	9
3.2.2 Linear Support Vector Machine	10
3.3 Training Process	10
3.3.1 Dataset Partitioning	10
3.3.2 Model Training Workflow	10
3.4 Evaluation Metrics	11
4.0 Apache System Architecture	12
4.1 System Overview Diagram	12
4.2 Implementation	14
4.2.1 Kafka Producer	14
4.2.2 Spark Structured Streaming	14
4.2.3 ElasticSearch Data Storage	15
4.2.4 Kibana Visualization	16
5.0 Analysis and Result	16

5.1 Sentiment Analysis Results	17
5.1.1 Actual and Predicted Sentiment Distribution	17
5.1.2 Prediction Outcome Analysis	18
5.1.3 Prediction Confidence	19
5.2 Real-Time Streaming Results.....	20
5.2.1 Real-Time Sentiment System	20
5.2.2 Sentiment Trend Over Time	21
5.3 Key Insights.....	22
5.3.1 Positive Review Analysis	22
5.3.2 Negative Review Analysis.....	23
6.0 Optimization and Comparison	24
6.1 Model Optimization	24
6.1.1 Naive Bayes	24
6.1.2 Linear Support Vector Machine (SVM).....	25
6.2 Model Performance Comparison	25
6.3 Pipeline Optimization.....	26
6.4 Streaming Pipeline Performance Evaluation.....	27
6.4.1 Processing Time by Micro-Batch	27
6.4.2 Records Processed by Micro-Batch.....	28
6.4.3 Throughput by Micro-Batch	28
6.4.4 Streaming Pipeline Performance Summary	29
7.0 Conclusion and Future Work	30
7.1 Conclusion.....	30
7.2 Future Work	31
8.0 References.....	32
9.0 Appendices.....	33

1.0 Introduction

This section introduces the background, problem statement, objectives, and scope of the project. It also provides an overview of the real-time sentiment analysis system and the technologies used throughout the project.

1.1 Project Background

The rapid growth of mobile applications has significantly increased the volume of user-generated reviews available on application marketplaces such as the Google Play Store. These reviews provide valuable feedback regarding application functionality, service quality, usability, and overall user satisfaction. For digital payment applications such as Touch 'n Go eWallet, customer reviews can reveal users' experiences, common issues, and suggestions for improvement. However, the continuously increasing number of reviews makes manual analysis inefficient, time-consuming, and difficult to perform in a timely manner.

Sentiment analysis is a Natural Language Processing (NLP) technique that automatically identifies the sentiment expressed in textual data by classifying it into positive, negative, or neutral categories. It has become one of the most widely adopted approaches for analysing customer opinions and supporting business decision-making (Liu, 2012). Combined with real-time data processing technologies, sentiment analysis enables organizations to continuously monitor customer opinions and respond more quickly to emerging issues. Apache Kafka provides reliable message streaming for handling continuous data ingestion, while Apache Spark Structured Streaming enables scalable and parallel processing of streaming data. Apache Spark is designed for high-performance distributed data processing, making it suitable for large-scale real-time analytics (Zaharia et al., 2016).

This project develops a real-time sentiment analysis pipeline for Touch 'n Go eWallet Google Play Store reviews. Customer reviews are collected from the Google Play Store, preprocessed using

Natural Language Processing techniques, and classified using machine learning models. The classified results are streamed through Apache Kafka and Apache Spark Structured Streaming before being stored in Elasticsearch and visualized using Kibana dashboards to provide real-time insights into customer sentiment.

1.2 Problem Statement

Touch 'n Go eWallet receives thousands of customer reviews through the Google Play Store, providing valuable information regarding user satisfaction and application performance. However, manually examining large volumes of customer reviews is labour-intensive and cannot provide timely insights into changing customer opinions.

Conventional batch processing approaches analyze data only after a collection period has been completed, resulting in delayed feedback that limits the ability to monitor customer sentiment continuously. In addition, raw customer reviews often contain noisy and unstructured text, including punctuation, special characters, and inconsistent formatting, which must be preprocessed before accurate sentiment classification can be performed.

Therefore, there is a need for a scalable real-time sentiment analysis system capable of continuously processing customer reviews, automatically classifying sentiment, and presenting updated analytical results through an interactive dashboard. Such a system can assist organizations in understanding customer feedback more efficiently and support faster decision-making for improving application quality and user experience.

1.3 Objectives

The main objective of this project is to develop a real-time sentiment analysis pipeline for Touch 'n Go eWallet customer reviews using Apache Kafka and Apache Spark Structured Streaming.

The specific objectives are:

1. To collect customer reviews of the Touch 'n Go eWallet application from the Google Play Store and preprocess the textual data using Natural Language Processing techniques.
2. To develop and compare the performance of Naive Bayes and Linear Support Vector Machine (SVM) models for sentiment classification.

3. To implement a real-time streaming pipeline using Apache Kafka and Apache Spark Structured Streaming for processing customer reviews.
4. To evaluate the performance of the Apache Spark Structured Streaming pipeline based on processing time, throughput, and streaming efficiency.
5. To store the classified sentiment results in Elasticsearch and develop an interactive Kibana dashboard for visualizing real-time sentiment analysis results and customer feedback trends.

1.4 Project Scope

This project focuses on customer reviews collected from the **Touch 'n Go eWallet** application available on the Google Play Store. Only English-language reviews from Malaysian users are considered for sentiment analysis. The collected reviews undergo several preprocessing steps, including text cleaning, tokenization, stop-word removal, and lemmatization before being transformed into numerical features using the TF-IDF technique.

Two machine learning algorithms, namely Naive Bayes and Linear Support Vector Machine (SVM), are trained and evaluated to identify the better-performing sentiment classification model. The selected model is integrated into a real-time streaming pipeline using Apache Kafka and Apache Spark Structured Streaming. The classified sentiment results are stored in Elasticsearch and visualized through Kibana dashboards, which display sentiment distribution, sentiment trends over time, prediction confidence, and frequently occurring positive and negative words.

2.0 Data Acquisition and Preprocessing

This section describes the process of collecting and preparing the raw dataset used for sentiment analysis. It includes the data source, the data collection method, the data preprocessing techniques applied to improve data quality, and the final dataset used for model development.

2.1 Data Source Description

The dataset used in this project contains user reviews collected from the Google Play Store for the Touch 'n Go eWallet mobile application. These reviews represent real user experiences and feedbacks about the application's features, usability, performance, and payment services. These

reviews provide useful information for identifying user satisfaction and common issues about the application, making them suitable for sentiment analysis.

The reviews were collected using the Python **google-play-scraper** library, which retrieves publicly available reviews directly from the Google Play Store. Table 2.1 shows the attributes extracted from the raw dataset.

Table 2.1 Raw Dataset Attributes

Attribute	Description
review_id	Unique identifier for each review
review_text	User's review text
rating	User's rating for the application
thumbs_up_count	Number of users who liked the review
review_datetime	Date and time when the review was posted
sentiment_label	The sentiment category generated from the rating score

2.2 Data Collection Method

The review data was collected using Python in a Jupyter Notebook environment. The **google-play-scraper** library was used to access the Google Play Store and retrieve user reviews for the Touch 'n Go eWallet application.

The data collection process was configured using the application's package name (APP_ID), language, country, review sorting method, and the number of reviews to retrieve. The following shows the configuration to retrieve user reviews:

- APP_ID = *"my.com.tngdigital.ewallet"*
- lang = *"en"*
- country = *"my"*
- sort = *Sort.NEWEST*
- count = *20000*

After the reviews were successfully retrieved, the data was converted into a Pandas DataFrame for further processing. Next, only a few attributes that are useful for sentiment analysis were retained. The retained attributes include the review_id, review_text, rating, thumbs_up_count, and

review_date. Then, the column names were renamed to more consistent and meaningful names to improve readability.

Next, a new attribute named **sentiment_label** was created, and the sentiment labels were generated automatically from the user's ratings using a rule-based approach. Reviews with ratings of 1 and 2 were labeled as **negative**, reviews with ratings of 3 were labelled as **neutral**, while ratings of 4 and 5 were labeled as **positive**.

Before saving the dataset, a basic quality check was performed to identify empty review text and duplicated review IDs. The reviews with empty review text were removed, and duplicated reviews were eliminated to ensure that the dataset was complete and unique. Finally, the dataset was exported as a CSV file named *touch_n_go_reviews_raw.csv* and served for further data preprocessing and model development.

2.3 Data Cleaning Steps

After the raw review dataset was collected, text processing was applied to clean and standardize the review text before model training. The data cleaning was performed using the **clean_text()** function.

First, the missing review text was handled by replacing null values with an empty string. Then, all texts were converted to lowercase to ensure words with different capitalization were treated consistently. Besides, English contractions were expanded using the **contractions** library, such as converting “can’t” into “cannot”.

Next, a text normalization dictionary was applied to handle Malaysian-style language. For example, short forms such as “tak”, “tk”, “x”, and “xde” were converted into standard Malay terms such as “tidak” and “tiada”. At the same time, common abbreviations such as “dpt”, “blh”, “sgt”, “pls”, and “plz” were also normalized.

Furthermore, the cleaning process also removed noisy elements, including URLs, email addresses, HTML tags, emojis or non-ASCII characters, repeated characters, numbers, punctuation marks, and extra spaces. After that, the text was split into tokens. English stop words and custom Malay/English stop words were removed. Moreover, words with fewer than three characters were also filtered out. Finally, lemmatization was applied using the WordNet Lemmatizer to convert words into their base form.

After that, the cleaned text was stored in a new column named **cleaned_review**. Empty cleaned reviews were removed from the dataset to ensure that only meaningful text remained for model training. Finally, the cleaned dataset was saved as *cleaned_data.csv*.

2.4 Final Dataset Description

The *cleaned_data.csv* act as the final cleaned dataset and was used as the input for feature extraction and sentiment model development. The final dataset contains the original review text, the cleaned review text, rating information, sentiment label, and several additional text length features. Table 2.2 shows the final dataset attributes.

Table 2.2 Final Dataset Attributes

Attribute	Description
review_id	Unique identifier for each review
review_text	User's review text
rating	User's rating for the application
thumbs_up_count	Number of users who liked the review
review_datetime	Date and time when the review was posted
sentiment_label	The sentiment category generated from the rating score
cleaned_review	User's review text after preprocessing
raw_review_length	Number of characters in the original review text
cleaned_review_length	Number of characters in the cleaned review text
cleaned_word_count	Number of words remaining in cleaned review text after preprocessing

3.0 Sentiment Model Development

Sentiment analysis is a Natural Language Processing (NLP) technique used to identify the emotional polarity expressed in textual data. In this project, sentiment analysis was performed on Touch 'n Go eWallet user reviews to classify each review into positive, neutral, or negative sentiment categories. The sentiment analysis model was developed through feature engineering, machine learning model selection, model training, and performance evaluation before being integrated into the Apache Spark streaming pipeline for real-time sentiment prediction.

3.1 Feature Engineering

Feature engineering is an essential process in text classification because machine learning algorithms cannot directly process raw textual data. Therefore, the collected reviews were first preprocessed to remove noise and improve text consistency before being transformed into numerical feature vectors using the TF-IDF technique.

3.1.1 Text Preprocessing

The cleaned review dataset stored in `cleaned_data.csv` was used for model development. Records containing missing values in either the `cleaned_review` or `sentiment_label` columns were removed before training. The preprocessing pipeline, implemented in `sentiment_preprocess.py`, performed several text cleaning operations, including lowercase conversion, contraction expansion, Malay text normalization, removal of URLs, HTML tags, emojis, punctuation, and numerical values. English stopwords from the NLTK library together with a customized Malay stopwords list were removed, while lemmatization was performed using the `WordNetLemmatizer`. The cleaned reviews stored in the `cleaned_review` column of `cleaned_data.csv` were subsequently used for model training.

These preprocessing steps reduced textual noise, standardized the review content, and improved the quality of the textual data for feature extraction and machine learning.

3.1.2 TF-IDF Feature Extraction

After preprocessing, the cleaned reviews were converted into numerical feature vectors using the Term Frequency–Inverse Document Frequency (TF-IDF) method. TF-IDF assigns higher weights to important words while reducing the influence of frequently occurring terms, allowing the classifier to better distinguish between different sentiment categories (Manning et al., 2008).

The project implemented Scikit-learn's `TfidfVectorizer` with the configuration shown in Table 3.1.

Table 3.1 TF–IDF Vectorizer Configuration

Parameter	Value
Maximum Features	8,000
N-gram Range	(1,2)
Minimum Document Frequency	2
Sublinear TF	True
Lowercase	False

The TF–IDF vectorizer was fitted using only the training dataset before transforming the validation and testing datasets. This approach prevents data leakage and ensures that the evaluation results reflect the model's ability to generalize unseen data (Manning et al., 2008).

3.2 Models Selection

To identify the most suitable sentiment classification model, two supervised machine learning algorithms were selected for sentiment classification to determine the most suitable model for deployment in the real-time streaming pipeline. Both models were trained using identical datasets and TF–IDF feature representations to ensure a fair and consistent comparison.

3.2.1 Multinomial Naive Bayes

Multinomial Naive Bayes is a probabilistic classification algorithm based on Bayes' Theorem and is widely used for text classification due to its efficiency in handling high-dimensional sparse data (Jurafsky & Martin, 2023). The algorithm estimates the probability that a review belongs to a particular sentiment class by assuming that each feature contributes independently to the prediction.

In this project, the default implementation of `MultinomialNB()` from the Scikit-learn library was used. The model was selected as one of the candidate classifiers because it offers fast training and prediction while achieving competitive performance for TF–IDF-based text classification.

3.2.2 Linear Support Vector Machine

Linear Support Vector Machine (Linear SVM) was implemented as a comparison model. Unlike Naive Bayes, Linear SVM is a discriminative classifier that determines the optimal hyperplane separating different sentiment classes by maximising the classification margin (Cortes & Vapnik,

1995). Linear SVM has demonstrated strong performance in many sentiment analysis applications involving sparse textual features.

The model was implemented using LinearSVC with `class_weight="balanced"` and `random_state=42` to improve robustness and ensure reproducible results. After training, its performance was compared with the Multinomial Naive Bayes classifier using the same evaluation metrics.

3.3 Training Process

The sentiment classification models were developed using a structured training workflow consisting of dataset partitioning, feature extraction, model training, and model serialization. This workflow ensured that the models could generalise effectively to unseen data while remaining suitable for deployment in the real-time streaming environment.

3.3.1 Dataset Partitioning

The cleaned review dataset was divided into training, validation, and testing sets using Scikit-learn's `train_test_split()` function with stratified sampling. The dataset was partitioned into 70% training, 10% validation, and 20% testing, while maintaining the original sentiment distribution across all subsets. A fixed random seed (`random_state = 42`) was applied to ensure reproducibility throughout the experiments.

The training dataset was used to build the machine learning models, the validation dataset was used for intermediate performance assessment, and the testing dataset was reserved for the final evaluation of the trained models.

3.3.2 Model Training Workflow

The training process began by fitting the TF-IDF vectorizer on the training dataset and transforming the validation and testing datasets using the same vocabulary. This ensured that all datasets shared a consistent feature of representation.

Subsequently, the Multinomial Naive Bayes and Linear Support Vector Machine classifiers were trained independently using the TF-IDF feature vectors. After training, both models generated predictions on the validation and testing datasets. The trained models were evaluated on the

validation and testing datasets using Accuracy, weighted Precision, weighted Recall, and weighted F1-score. Confusion matrices were also generated using the Scikit-learn library and visualized with Matplotlib and Seaborn to analyze the classification performance across the three sentiment classes.

Finally, the trained TF-IDF vectorizer and both machine learning models were saved using the Joblib library as `vectorizer.pkl`, `naive_bayes_model.pkl`, and `linear_svm_model.pkl`. These serialized files were subsequently loaded by the Apache Spark streaming application to perform real-time sentiment prediction without retraining the model. In this project, the default hyperparameters of the selected machine learning algorithms were used. No automated hyperparameter tuning techniques, such as Grid Search or Cross-Validation, were implemented. The models were instead compared based on their performance using the testing dataset.

3.4 Evaluation Metrics

The performance of the sentiment classification models was evaluated using a held-out testing dataset generated from the stratified 70:10:20 train-validation-test split. To provide a comprehensive assessment of model performance, four widely used classification metrics were employed: Accuracy, Precision, Recall, and F1-score (Sokolova & Lapalme, 2009).

Accuracy measures the proportion of correctly classified reviews out of the total number of predictions, providing an overall indication of classification performance. However, accuracy alone may not adequately reflect model performance when the distribution of sentiment classes is imbalanced. Therefore, weighted Precision, weighted Recall, and weighted F1-score were also calculated to provide a more balanced evaluation across all sentiment categories.

Precision measures the proportion of correctly predicted instances among all instances predicted for a particular sentiment class, while Recall measures the proportion of correctly identified instances among all actual instances belonging to that class. The F1-score, which is the harmonic mean of Precision and Recall, provides a balanced measure of classification performance by considering both false positive and false negative predictions. In this project, the weighted F1-score was used as the primary evaluation metric because it accounts for the class distribution and provides a more reliable measure of overall model performance.

In addition to these numerical metrics, confusion matrices were generated for both classification models using the Scikit-learn library and visualized with Matplotlib and Seaborn. A confusion matrix illustrates the distribution of correct and incorrect predictions for each sentiment class (positive, neutral, and negative), allowing the classification performance of each model to be analysed in greater detail.

The evaluation metrics described in this section were subsequently used to compare the performance of the developed sentiment classification models. The comparison results and the selection of the final production model are presented in Chapter 6, Optimization and Comparison.

4.0 Apache System Architecture

4.1 System Overview Diagram

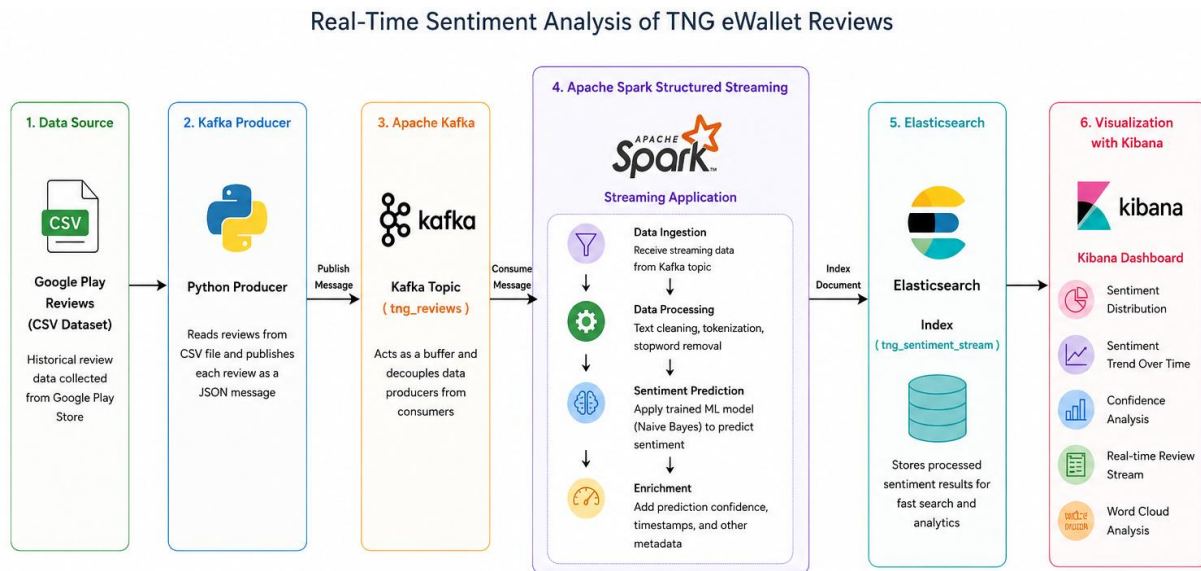


Figure 4.1 Overall System Architecture for Real-Time Sentiment Analysis of TNG eWallet Reviews

Figure 4.1 illustrates the overall architecture of the real-time sentiment analysis system developed for Touch 'n Go eWallet customer reviews. The system integrates Apache Kafka, Apache Spark Structured Streaming, Elasticsearch, and Kibana to provide an end-to-end streaming analytics pipeline.

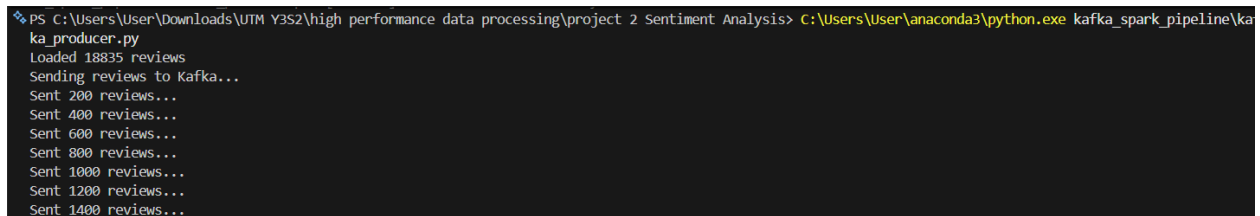
The process begins with the Google Play review dataset, where customer reviews are continuously published by the Kafka Producer into a Kafka topic. Apache Kafka serves as the messaging platform that buffers and distributes streaming data between independent system components, allowing producers and consumers to operate asynchronously.

Apache Spark Structured Streaming continuously consumes review messages from the Kafka topic and performs real-time data processing. During this stage, each review undergoes text preprocessing before the trained Naive Bayes sentiment classification model is applied to predict the review sentiment. The processing pipeline also generates additional metadata, including prediction confidence scores and processing timestamps.

After the sentiment prediction is completed, the processed records are indexed into Elasticsearch, which functions as the storage and search engine for streaming data. Finally, Kibana retrieves the indexed documents from Elasticsearch and presents the processed sentiment information through interactive dashboards. This architecture enables continuous data ingestion, scalable stream processing, efficient data storage, and near real-time visualization of customer sentiment.

4.2 Implementation

4.2.1 Kafka Producer



```
PS C:\Users\User\Downloads\UTM Y3S2\high performance data processing\project 2 Sentiment Analysis> C:\Users\User\anaconda3\python.exe kafka_spark_pipeline\ka_producer.py
Loaded 18835 reviews
Sending reviews to Kafka...
Sent 200 reviews...
Sent 400 reviews...
Sent 600 reviews...
Sent 800 reviews...
Sent 1000 reviews...
Sent 1200 reviews...
Sent 1400 reviews...
```

Figure 4.2.1 Kafka Producer Sending Review Messages

The Kafka Producer is responsible for continuously sending review data into the Apache Kafka messaging system for real-time processing. In this project, the producer reads the cleaned Touch 'n Go eWallet review dataset from a CSV file and publishes each review as a JSON message to the designated Kafka topic. To simulate a real-time streaming environment, a short delay is introduced between each message transmission. This allows the incoming data to be processed incrementally rather than all at once.

Using Kafka as the ingestion layer provides reliable and scalable data streaming by decoupling the data source from downstream processing components. Each published message contains the

review text, cleaned review, rating, review date, and actual sentiment label, ensuring that complete review information is available for downstream processing.

4.2.2 Spark Structured Streaming

```
>> C:\Users\User\anaconda3\Scripts\spark-submit.cmd --packages org.apache.spark:spark-sql-kafka-0-10_2.13:4.1.1 kafka_spark_pipeline\spark_streaming.py
Spark streaming started.
Waiting for Kafka messages...
Press Ctrl+C to stop.
Batch 1: processed 72 records | time=4.71s | throughput=15.28 records/sec
Batch 2: processed 105 records | time=5.60s | throughput=18.76 records/sec
Batch 3: processed 125 records | time=6.46s | throughput=19.34 records/sec
Batch 4: processed 136 records | time=6.89s | throughput=19.73 records/sec
Batch 5: processed 145 records | time=7.48s | throughput=19.38 records/sec
Batch 6: processed 157 records | time=8.00s | throughput=19.63 records/sec
```

Figure 4.2.2 Spark Structured Streaming Processing Kafka Messages

The Spark Streaming Consumer is responsible for consuming review messages from the Kafka topic and performing real-time sentiment analysis. Apache Spark Structured Streaming continuously monitors the Kafka topic and processes incoming review records in micro-batches. For each incoming review, the application performs data preprocessing, applies the trained Naive Bayes model to predict sentiment, calculates the prediction confidence score, and enriches the record with additional metadata.

During execution, the application reports processing statistics such as the number of processed records, execution time, and throughput for each micro-batch.

Apache Spark was selected because it provides fault-tolerant distributed processing and supports scalable real-time analytics through Structured Streaming. Its seamless integration with Kafka and Elasticsearch makes it suitable for building an end-to-end streaming sentiment analysis pipeline.

4.2.3 Elasticsearch Data Storage



```
{
  "took": 3,
  "timed_out": false,
  "_shards": {
    "total": 1,
    "successful": 1,
    "skipped": 0,
    "failed": 0
  },
  "hits": {
    "total": {
      "value": 10000,
      "relation": "gte"
    },
    "max_score": 1.0,
    "hits": [
      {
        "_index": "tng_sentiment_stream",
        "_id": "oa33Np8B3HK9r7z4KHTL",
        "_score": 1.0,
        "_source": {
          "review_id": "3f30b3af-c449-46e7-aac9-0f6aa108450c",
          "review_text": "good luck",
          "cleaned_review": "good luck",
          "rating": 5,
          "actual_sentiment": "positive",
          "predicted_sentiment": "positive",
          "prediction_confidence": 0.9802817149171951,
          "review_datetime": "2026-05-24 13:28:06",
          "processed_at": "2026-07-06T09:36:47.105205+00:00",
          "source": "spark_streaming"
        }
      },
      {
        "_index": "tng_sentiment_stream",
        "_id": "oq33Np8B3HK9r7z4KHT7",
        "_score": 1.0,
        "_source": {
          "review_id": "55598aa0-874a-4c7a-9ec9-cfa787439053",
          "review_text": "Nice",
          "cleaned_review": "nice",
          "rating": 5,
          "actual_sentiment": "positive",
          "predicted_sentiment": "positive",
          "prediction_confidence": 0.9916781187287524,
          "review_datetime": "2026-05-24 13:06:41"
        }
      }
    ]
  }
}
```

Figure 4.2.3 Sample Indexed Sentiment Documents

Elasticsearch functions as the storage layer for the real-time sentiment analysis pipeline. After the Spark Streaming Consumer processes each incoming review, the enriched sentiment record is indexed into the `tng_sentiment_stream` index. Each indexed document contains both the original review information and the generated prediction results, including the predicted sentiment, prediction confidence score, review rating, review timestamp, processing timestamp, and cleaned review text.

Elasticsearch provides efficient indexing, distributed storage, and fast search capabilities for streaming sentiment data. These indexed documents form the data source used for subsequent analysis and visualization.

4.2.4 Kibana Visualization

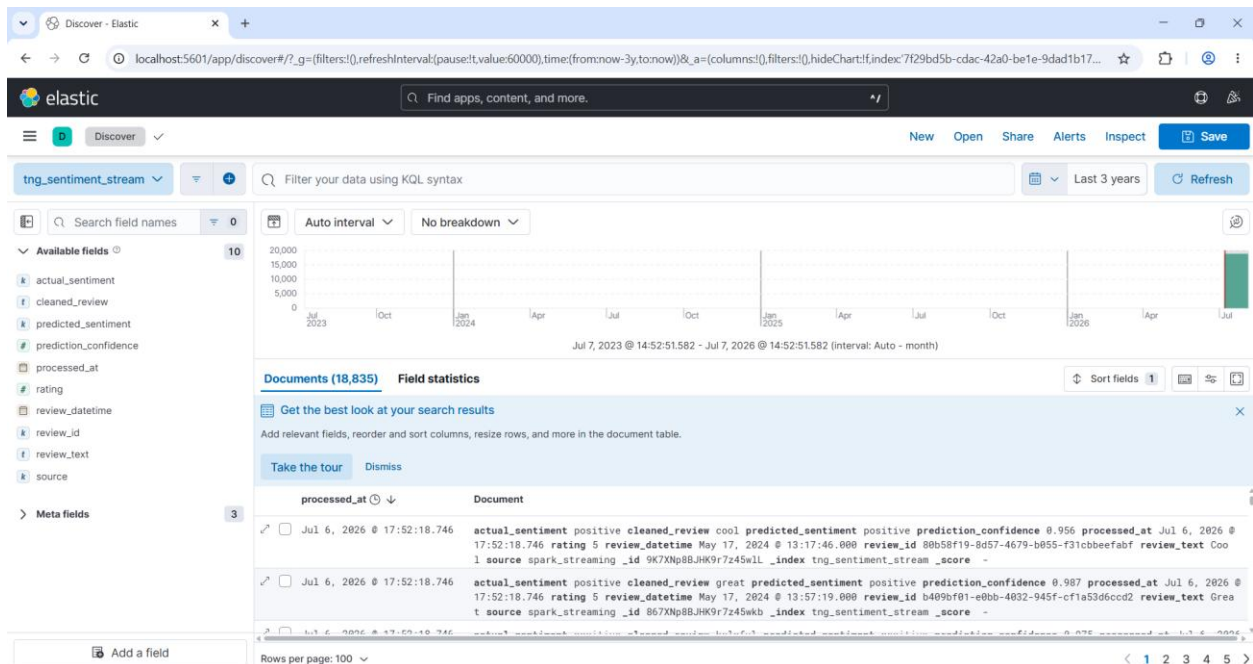


Figure 4.2.4 Kibana Discover Interface Displaying Indexed Sentiment Documents

Kibana provides an interactive interface for exploring the sentiment data stored in Elasticsearch. Through the Discover feature, users can inspect individual review records, verify prediction results, search specific documents, and apply filters based on available fields.

Figure 4.2.4 shows the Discover interface displaying documents from the `tng_sentiment_stream` index. Each record contains the original review together with the predicted sentiment, prediction confidence, review rating, review timestamp, and processing timestamp. This interface was used to verify that the streaming pipeline correctly stored sentiment prediction results before generating analytical dashboards.

5.0 Analysis and Result

This section presents the results produced by the real-time sentiment analysis system. After the trained Naive Bayes model was successfully integrated with Apache Kafka, Spark Structured Streaming, Elasticsearch, and Kibana, incoming Touch 'n Go eWallet reviews from Google Play Store were automatically classified into sentiment categories and visualized through an interactive dashboard. The section discusses the sentiment classification results, demonstrates the real-time

monitoring capability of the system, and summarizes the key insights obtained from the analysis of user reviews.

5.1 Sentiment Analysis Results

This section presents the sentiment analysis results produced by the deployed sentiment classification model. After each review was processed by the streaming pipeline, the predicted sentiment, prediction confidence, and review information were stored in Elasticsearch and visualized through Kibana dashboards.

5.1.1 Actual and Predicted Sentiment Distribution

Figure 5.1 illustrates the distribution of the actual sentiment labels generated from user ratings, while Figure 5.2 presents the sentiment distribution predicted by the Naive Bayes model. The actual dataset consists of **68.66% positive**, **27.46% negative**, and **3.87% neutral** reviews. The predicted results contain **68.56% positive** and **31.44% negative** reviews.

The predicted distribution closely matches the actual proportion of positive reviews, indicating that the model successfully captures the overall sentiment trend of the dataset. However, the neutral class is classified into either positive or negative categories, resulting in the absence of the neutral class in the prediction results. This behavior is expected because the neutral class represents only a small proportion of the dataset, thus having fewer training examples.

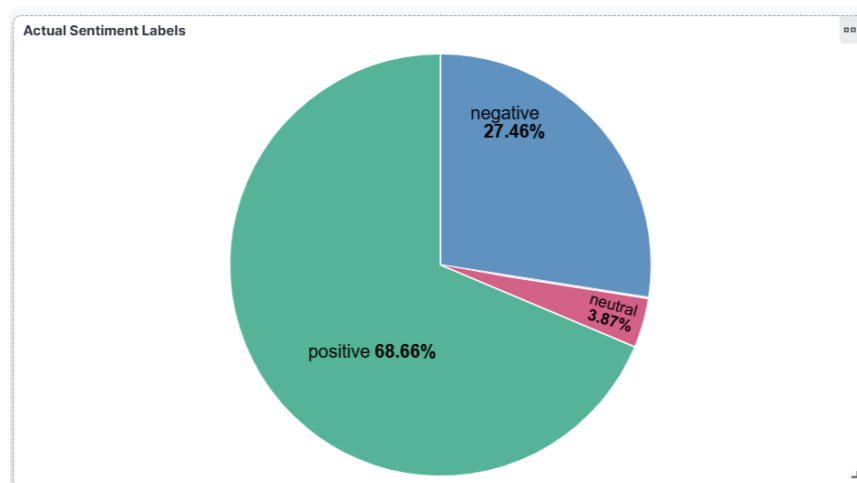


Figure 5.1 Actual Sentiment Labels

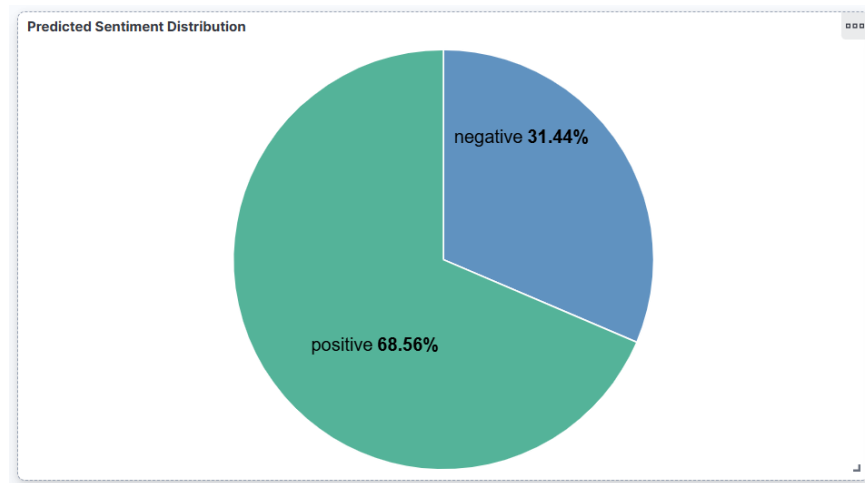


Figure 5.2 Predicted Sentiment Distribution

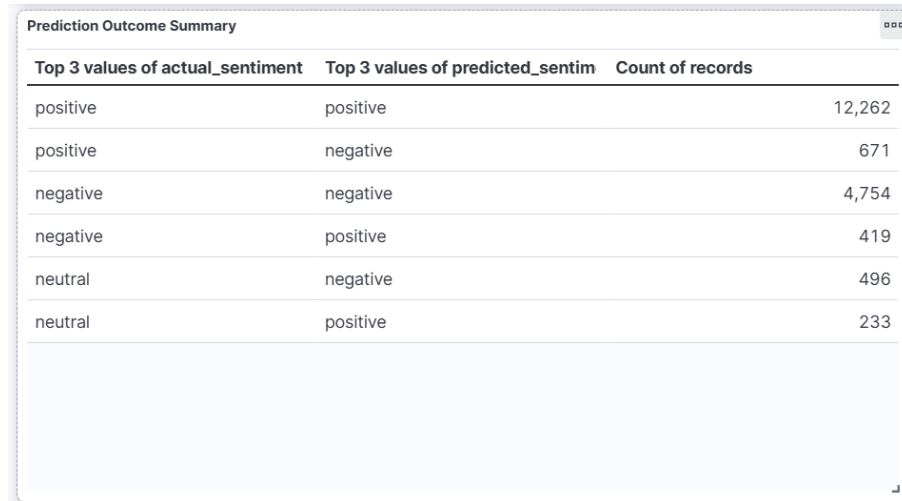
5.1.2 Prediction Outcome Analysis

Figure 5.3 summarizes the relationship between the actual sentiment labels and the predicted sentiment labels generated by the model.

Most positive reviews (**12,262 reviews**) were correctly classified as positive, while **4,754 negative reviews** were correctly classified as negative. A smaller number of positive reviews (**671 reviews**) were incorrectly classified as negative, whereas **419 negative reviews** were predicted as positive.

Besides, the neutral reviews were classified as either negative (**496 reviews**) or positive (**233 reviews**). This indicates that the model finds it difficult to distinguish neutral reviews because they often contain mixed opinions and represent only a small percentage of the training data.

Overall, the prediction outcome indicates that the model performs well in distinguishing positive and negative sentiments while showing lower performance for the minority neutral class.



Top 3 values of actual_sentiment	Top 3 values of predicted_sentim	Count of records
positive	positive	12,262
positive	negative	671
negative	negative	4,754
negative	positive	419
neutral	negative	496
neutral	positive	233

Figure 5.3 Prediction Outcome Summary

5.1.3 Prediction Confidence

Figure 5.4 shows the average prediction confidence of the deployed Naive Bayes model is **0.894**, indicating that the model is generally confident in its sentiment predictions.

Figure 5.5 further shows the average prediction confidence for each rating level. Reviews with higher ratings, especially reviews with rating of 5, have the highest prediction confidence, whereas reviews with rating of 3 show the lowest confidence. This observation is reasonable because reviews with rating of 5 usually express clear positive opinions, while reviews with rating of 3 often contain mixed or neutral opinions that are more difficult to classify.

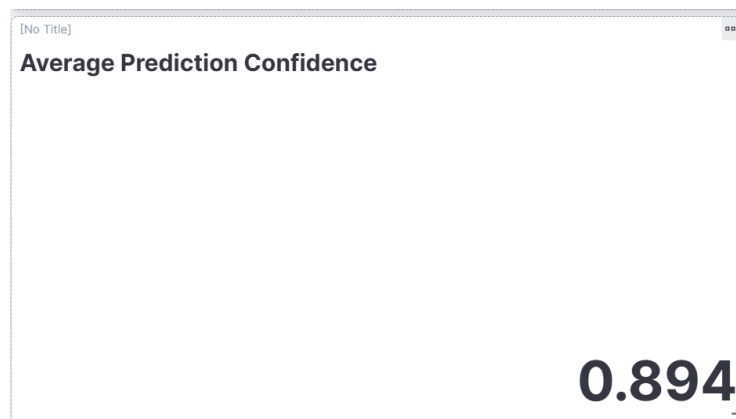


Figure 5.4 Average Prediction Confidence

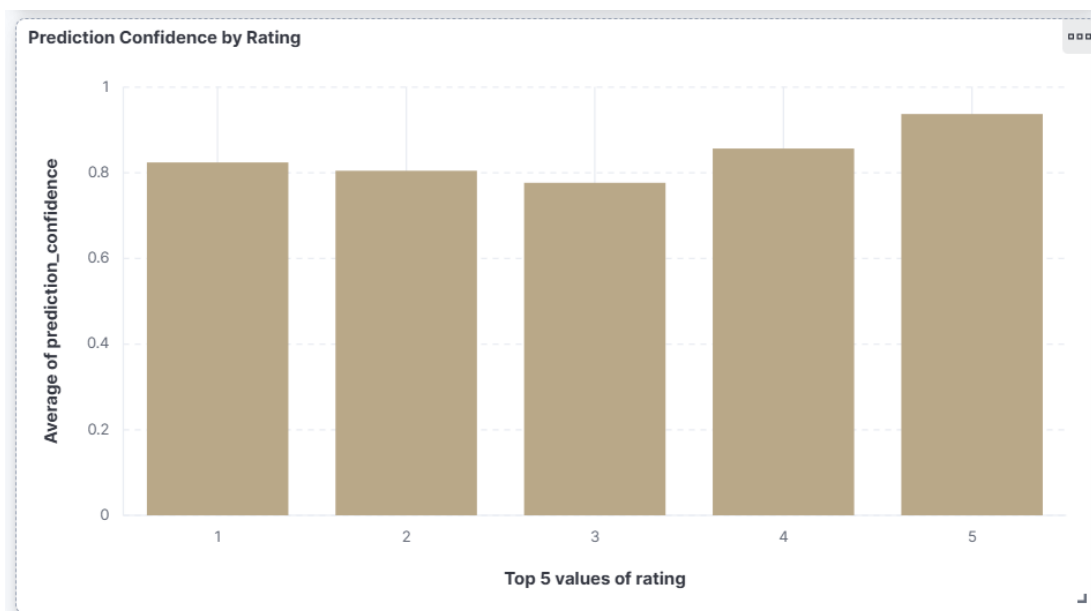


Figure 5.5 Prediction Confidence by Rating

5.2 Real-Time Streaming Results

One of the main objectives of this project is to perform sentiment analysis in real time. Apache Kafka continuously streams incoming reviews to Spark Structured Streaming, where the trained Naive Bayes model predicts the sentiment label before storing the results in Elasticsearch. Kibana then visualizes the processed data using an interactive dashboard.

5.2.1 Real-Time Sentiment System

Figure 5.6 displays the processed reviews stored in Elasticsearch. Each record contains the processing timestamp, original review text, predicted sentiment, prediction confidence, and user rating. The dashboard is automatically updated whenever new reviews are processed, allowing users to monitor customer feedback without manual intervention.

Real-Time Sentiment Stream

18,835 documents

Columns 5

Sort fields 1

	processed_at	review_text	predicted_sentiment	prediction_confidence	rating
	Jul 6, 2026 @ 17:52:18.746	Cool	positive	0.956	5
	Jul 6, 2026 @ 17:52:18.746	Great	positive	0.987	5
	Jul 6, 2026 @ 17:52:18.746	more helpful	positive	0.975	5
	Jul 6, 2026 @ 17:52:18.746	why cant i make a loan for the fort...	negative	0.71	1

Rows per page: 100

<

1

2

3

4

5

>

Figure 5.6 Real-Time Sentiment Stream Dashboard

5.2.2 Sentiment Trend Over Time

Figure 5.7 presents the monthly sentiment trend of Touch 'n Go eWallet reviews. Positive reviews consistently dominate throughout the observation period, indicating that most users have a favorable opinion of the application. Negative reviews remain significantly lower than positive reviews, while neutral reviews account for only a small proportion of the total reviews.

The visualization enables organizations to identify changes in customer sentiment over time and detect periods with increasing negative feedback, allowing timely investigation and early corrective actions.

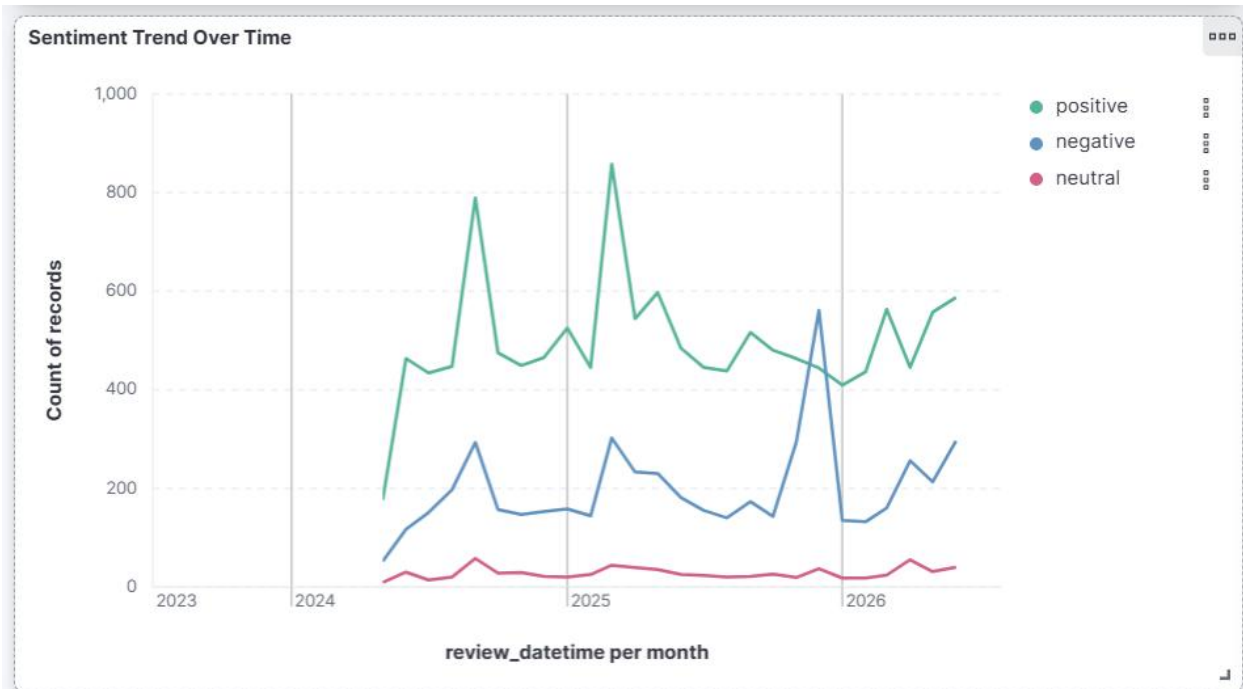


Figure 5.7 Sentiment Trend Over Time

5.3 Key Insights

The sentiment analysis results provide several useful insights into user perceptions of the Touch 'n Go eWallet application.

5.3.1 Positive Review Analysis

The positive word cloud in Figure 5.8 highlights the most frequently occurring words in positive reviews. Terms such as **convenient**, **useful**, **transaction**, **fast**, **bill**, **parking**, and **friendly** appear noticeably, indicating that users appreciate the application's convenience, speed, and ease of use. These words suggest that customers are generally satisfied with the application's digital payment and money transfer services.

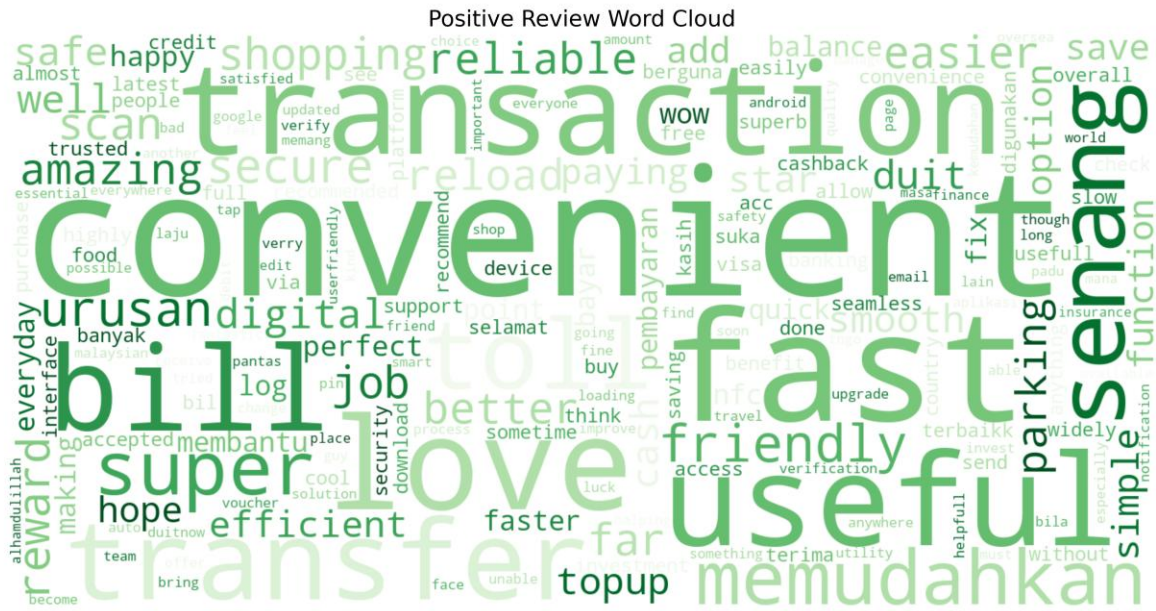


Figure 5.8 Positive Review Word Cloud

5.3.2 Negative Review Analysis

The negative word cloud in Figure 5.9 mentions the most common issues found by dissatisfied users. Frequently occurring words include **bad**, **transaction**, **verification**, **notification**, **balance**, **error**, and **slow**. These keywords indicate that many complaints are related to transaction failures, verification problems, and application performance issues.

Understanding these recurring issues can help developers prioritize system improvements and enhance the overall user experience.



Figure 5.9 Negative Review Word Cloud

6.0 Optimization and Comparison

This section presents the optimization techniques applied to improve the performance of the sentiment analysis system and compares the effectiveness of the developed models and processing approaches. It discusses the optimization of the selected sentiment classification models, evaluates their predictive performance, describes the optimization of the real-time streaming pipeline, and compares the performance of batch processing and streaming processing based on processing efficiency and throughput.

6.1 Model Optimization

This section discusses the sentiment classification models used in this project and the reasons for selecting them. Two machine learning algorithms, namely Naive Bayes and Linear Support Vector Machine (SVM), were developed and evaluated to determine the most suitable model for sentiment classification. Both models were trained using the TF-IDF feature representation generated from the preprocessed Touch 'n Go eWallet customer reviews.

6.1.1 Naive Bayes

Naive Bayes is a probabilistic machine learning algorithm based on Bayes' theorem that assumes feature independence between words. Due to its simplicity and computational efficiency, Naive Bayes is widely applied in text classification and sentiment analysis tasks. The algorithm performs particularly well when combined with TF-IDF features because it can efficiently process high-dimensional text data while maintaining relatively fast training and prediction times.

In this project, the Multinomial Naive Bayes classifier was implemented as one of the baseline sentiment classification models. Its low computational complexity makes it suitable for processing large volumes of textual data and provides an effective benchmark for evaluating more advanced classification algorithms.

6.1.2 Linear Support Vector Machine (SVM)

Linear Support Vector Machine (SVM) is a supervised machine learning algorithm that identifies the optimal hyperplane to separate different sentiment classes while maximizing the margin between them. Linear SVM has been widely adopted for sentiment analysis because of its ability to handle high-dimensional and sparse text features generated through TF-IDF vectorization.

Compared with probabilistic classifiers, Linear SVM generally provides stronger generalization capability and is more effective in distinguishing sentiment classes with complex decision boundaries. Therefore, Linear SVM was selected as the second classification model to evaluate whether it could provide improved sentiment prediction performance compared with Naive Bayes.

6.2 Model Performance Comparison

The performance of the Naive Bayes and Linear Support Vector Machine (SVM) models was evaluated using accuracy, precision, recall, and F1-score. These evaluation metrics provide a comprehensive assessment of each model's sentiment classification capability. Table 6.1 presents the comparison between the two sentiment classification models.

Table 6.1 Model Performance Comparison

Model	Accuracy	Precision	Recall	F1-score
Naive Bayes	88.93%	86.01%	88.93%	87.35%
Linear SVM	86.84%	86.18%	86.84%	86.43%

Based on the evaluation results, the Naive Bayes model achieved higher performance than the Linear SVM model across all evaluation metrics. It obtained the highest accuracy of **88.93%**, together with superior weighted precision, recall, and F1-score. These results indicate that the Naive Bayes classifier provides more accurate and consistent sentiment predictions for the Touch 'n Go eWallet review dataset used in this project.

Therefore, the Naive Bayes model was selected as the final sentiment classification model and integrated into the Apache Spark Structured Streaming pipeline for real-time sentiment prediction.

6.3 Pipeline Optimization

To improve the efficiency and reliability of the real-time sentiment analysis system, several optimization techniques were incorporated into the streaming pipeline. Apache Kafka was used as the distributed messaging platform to continuously stream customer reviews into the processing pipeline, while Apache Spark Structured Streaming processed the incoming data through micro-batch execution. This architecture enables continuous sentiment prediction while maintaining efficient resource utilization.

To reduce computational overhead, the trained TF-IDF vectorizer and Naive Bayes classification model were loaded only once during the initialization of the Spark streaming application and reused throughout the execution process. This approach eliminates repeated model loading for each micro-batch, thereby reducing prediction latency and improving overall processing efficiency.

Furthermore, Spark checkpointing was enabled to provide fault tolerance by preserving the streaming application's state information. In the event of system interruptions or failures, checkpointing allows the streaming pipeline to recover and resume processing with minimal data loss. The classified sentiment results were then stored in Elasticsearch, which supports efficient indexing and rapid retrieval of prediction results for dashboard visualization.

Overall, these optimization techniques improved the stability, efficiency, and reliability of the streaming pipeline, enabling continuous real-time sentiment analysis with consistent processing performance.

6.4 Streaming Pipeline Performance Evaluation

This section evaluates the performance of the Apache Spark Structured Streaming pipeline developed for real-time sentiment analysis of Touch 'n Go eWallet customer reviews. The evaluation focuses on three performance metrics: processing time, the number of records processed, and throughput across multiple Spark micro-batches. These metrics provide insight into the efficiency, stability, and scalability of the streaming pipeline during continuous data processing.

6.4.1 Processing Time by Micro-Batch

Figure 6.1 illustrates the processing time of each Spark Structured Streaming micro-batch throughout the streaming experiment.

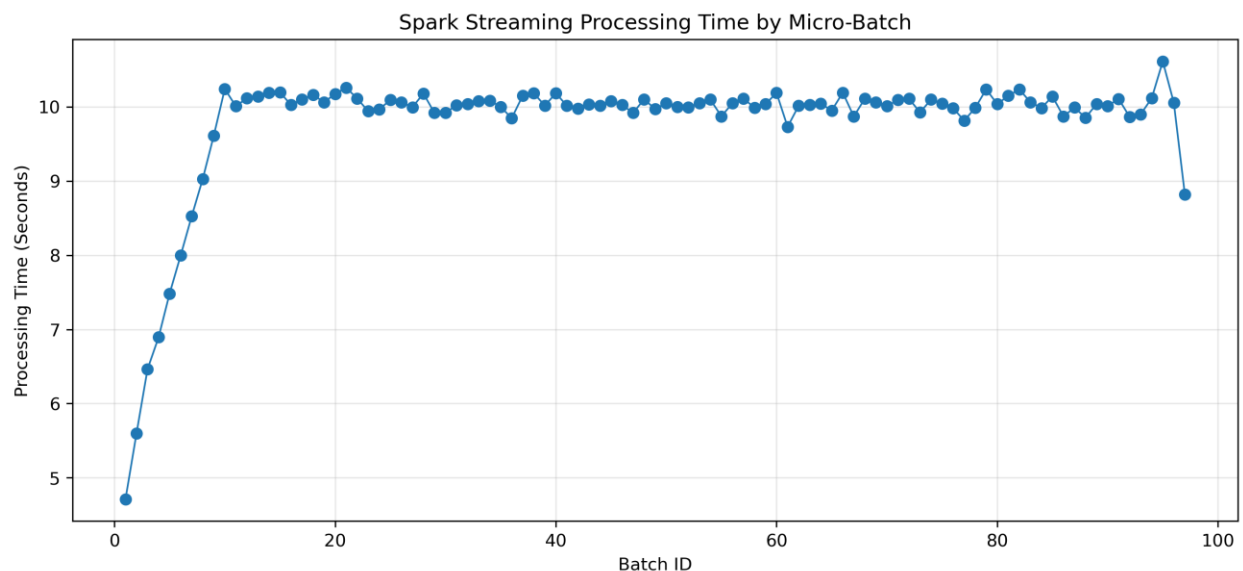


Figure 6.1 Spark Streaming Processing Time by Micro-Batch

As shown in Figure 6.1, the processing time increased gradually during the initial micro-batches, starting from approximately **4.7 seconds** before stabilizing at around **10 seconds** per micro-batch. The increase observed during the early stages was primarily due to the initialization of the Spark Structured Streaming application, including loading the machine learning model, establishing the Kafka connection, and allocating system resources. Once the initialization process was completed,

the processing time remained relatively stable with only minor fluctuations. This demonstrates that the streaming pipeline maintained consistent processing latency throughout the experiment and was capable of continuously processing incoming review data efficiently.

6.4.2 Records Processed by Micro-Batch

Figure 6.2 presents the number of customer reviews processed in each Spark micro-batch.

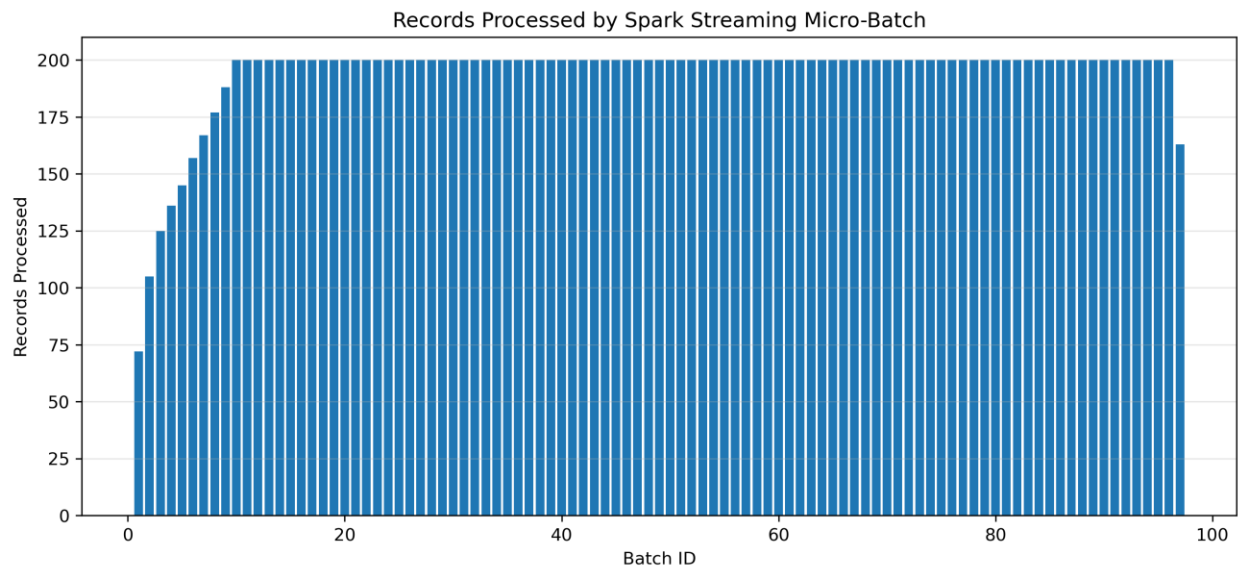


Figure 6.2 Records Processed by Spark Streaming Micro-Batch

Based on Figure 6.2, the number of records processed increased progressively during the initial batches before reaching the predefined batch size of **200 records**. The majority of the streaming process maintained a consistent workload of 200 records per micro-batch, indicating stable data ingestion from Apache Kafka and efficient processing by Apache Spark Structured Streaming. The final micro-batch processed fewer records because the remaining reviews in the dataset were less than the configured batch size. Overall, the results demonstrate that the streaming pipeline was capable of maintaining a consistent processing workload throughout the evaluation.

6.4.3 Throughput by Micro-Batch

Figure 6.3 illustrates the throughput achieved by each Spark Structured Streaming micro-batch, measured in records processed per second.

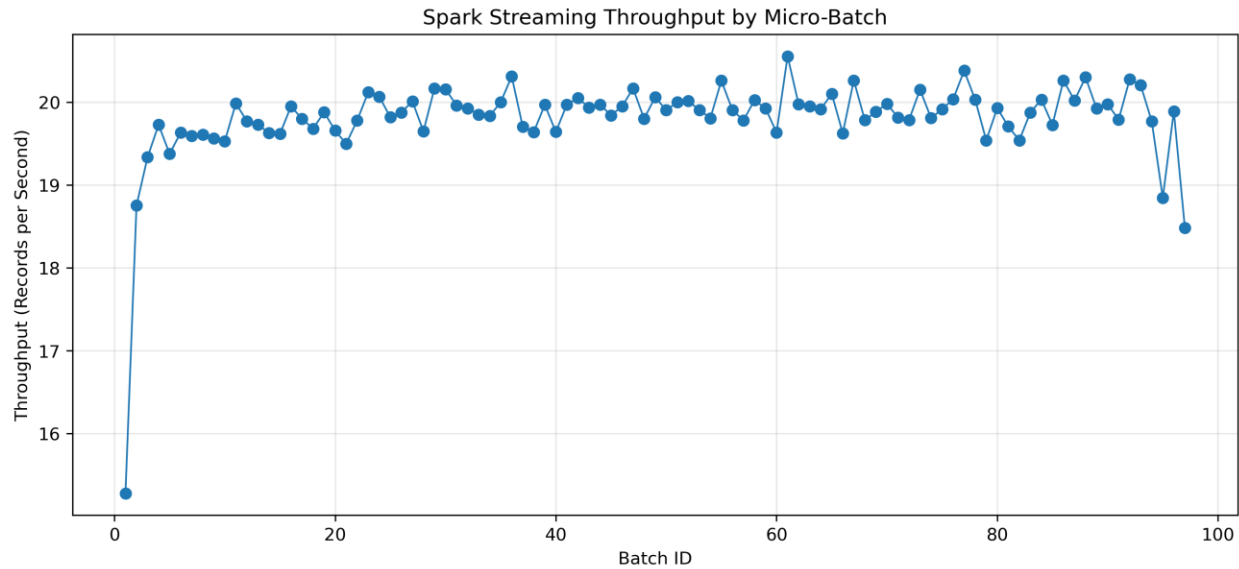


Figure 6.3 Spark Streaming Throughput by Micro-Batch

As illustrated in Figure 6.3, the throughput increased rapidly during the initialization phase before stabilizing at approximately **20 records per second** for most of the streaming process. Minor fluctuations were observed between individual micro-batches due to system scheduling and processing overhead. Nevertheless, the throughput remained relatively consistent, indicating that the streaming pipeline efficiently handled the incoming data stream while maintaining stable processing performance. The consistent throughput demonstrates the capability of Apache Spark Structured Streaming to process customer reviews continuously with minimal performance degradation.

6.4.4 Streaming Pipeline Performance Summary

Table 6.2 summarizes the overall performance of the streaming pipeline based on the experimental results.

Table 6.2 Streaming Pipeline Performance Summary

Performance Metric	Value
Total Records Processed	18,835
Number of Micro-Batches	97
Average Batch Size	194.18 records
Total Processing Time	949.38 seconds

Average Processing Time per Batch	9.79 seconds
Average Throughput	19.81 records/second
Overall Throughput	19.84 records/second
Minimum Throughput	15.28 records/second
Maximum Throughput	20.56 records/second

The streaming pipeline successfully processed **18,835 customer reviews** across **97 micro-batches**, with an average batch size of **194.18 records**. The total processing time was **949.38 seconds**, corresponding to an average processing time of **9.79 seconds per micro-batch**. Furthermore, the pipeline achieved an average throughput of **19.81 records per second**, with throughput ranging between **15.28** and **20.56 records per second** throughout the evaluation.

Overall, the experimental results indicate that the Apache Spark Structured Streaming pipeline achieved stable and efficient real-time processing performance. After the initial startup phase, both the processing time and throughput remained relatively consistent across successive micro-batches. These findings demonstrate that the developed streaming pipeline is capable of continuously processing customer reviews with reliable performance, making it suitable for real-time sentiment analysis applications.

7.0 Conclusion and Future Work

7.1 Conclusion

This project successfully developed a real-time sentiment analysis system for Touch 'n Go eWallet customer reviews by integrating Natural Language Processing (NLP), machine learning, and big data streaming technologies. The proposed system combines text preprocessing, TF-IDF feature extraction, sentiment classification, Apache Spark Structured Streaming, Apache Kafka,

Elasticsearch, and Kibana to enable continuous sentiment prediction and real-time visualization of customer feedback.

During the model development stage, two machine learning algorithms, namely Multinomial Naive Bayes and Linear Support Vector Machine (Linear SVM), were developed and evaluated using the same preprocessed dataset and TF–IDF feature representation. Based on the experimental results, the Multinomial Naive Bayes classifier achieved the best overall performance and was selected as the final production model for integration into the streaming pipeline.

The developed streaming pipeline successfully processed customer reviews in real time by continuously consuming messages from Apache Kafka, performing sentiment prediction using the trained machine learning model, and storing the prediction results in Elasticsearch for visualization through Kibana. Experimental evaluation demonstrated stable processing performance with an average throughput of approximately **19.8 records per second**, indicating that the proposed system can perform reliable and continuous sentiment analysis on streaming customer review data.

Overall, the project achieved its objectives by developing an end-to-end sentiment analysis system that combines effective text preprocessing, efficient machine learning classification, and scalable real-time data processing. The proposed solution demonstrates the feasibility of integrating machine learning with modern streaming technologies to support timely analysis of customer feedback and provide valuable insights for decision-making.

7.2 Future Work

Although the developed system achieved satisfactory performance, several improvements can be considered in future work to further enhance its accuracy, scalability, and maintainability.

Firstly, the sentiment classification performance can be improved through systematic hyperparameter optimization using techniques such as Grid Search or Randomized Search. Optimizing parameters for both the TF–IDF vectorizer and machine learning models may further improve classification accuracy and weighted F1-score.

Secondly, future studies may evaluate transformer-based language models, such as DistilBERT or multilingual BERT, to determine whether contextual language representations can improve sentiment classification performance compared with traditional TF–IDF-based approaches.

In addition, the training dataset can be expanded by incorporating a larger and more diverse collection of customer reviews collected over longer periods. This would improve the robustness and generalization capability of the sentiment classification model while providing a more representative reflection of real-world customer opinions.

From a deployment perspective, the streaming application can be further enhanced by deploying the Apache Spark pipeline in a distributed cluster environment instead of local execution. This would improve scalability and enable the system to process significantly larger volumes of streaming data. Furthermore, implementing automated model monitoring and periodic retraining would allow the sentiment classifier to adapt to changes in customer language and review patterns over time.

Overall, these improvements would further enhance the robustness, scalability, and practical applicability of the proposed real-time sentiment analysis system for future industrial deployment. Furthermore, the integration of Apache Kafka, Apache Spark Structured Streaming, and Elasticsearch demonstrates the effectiveness of combining machine learning with modern big data technologies to support scalable real-time sentiment analysis applications.

8.0 References

Aggarwal, C. C. (2018). *Machine learning for text*. Springer. <https://doi.org/10.1007/978-3-319-73531-3>

Cortes, C., & Vapnik, V. (1995). *Support-vector networks*. *Machine Learning*, 20(3), 273–297. <https://doi.org/10.1007/BF00994018>

Jurafsky, D., & Martin, J. H. (2023). *Speech and Language Processing* (3rd ed., draft). <https://web.stanford.edu/~jurafsky/slp3>

Liu, B. (2012). *Sentiment Analysis and Opinion Mining*. Morgan & Claypool Publishers.
<https://www.cs.uic.edu/~liub/FBS/SentimentAnalysis-and-OpinionMining.pdf>

Manning, C. D., Raghavan, P., & Schütze, H. (2008). *Introduction to information retrieval*. Cambridge University Press. <https://doi.org/10.1017/CBO9780511809071>

Sokolova, M., & Lapalme, G. (2009). *A systematic analysis of performance measures for classification tasks*. *Information Processing & Management*, 45(4), 427–437.
<https://doi.org/10.1016/j.ipm.2009.03.002>

Zaharia, M., Xin, R. S., Wendell, P., Das, T., Armbrust, M., Dave, A., ... Stoica, I. (2016). Apache Spark: A unified engine for big data processing. *Communications of the ACM*, 59(11), 56–65. <https://doi.org/10.1145/2934664>

9.0 Appendices

Link to the repository: <https://github.com/sean-seah/HPDP/tree/main/2526/project/p2/submission/ABC>